Optimizing Adaptive Context in Hierarchy Aware Graph Neural Networks

Chenjie Yin
Xi'an Jiaotong University
2244413949@stu.xjtu.edu.cn

***Abstract*—Graph neural networks (GNNs) are effective for learning relational data. However, standard GNNs rely on local message passing that cannot capture long range hierarchical context, while unconstrained expansion introduces noise and scalability issues. We study an optimized Adaptive Hierarchy aware Graph Neural Network (AHGNN) that pre serves direction during hierarchy estimation, constrains topology expansion with depth limited Breadth First Search (BFS) and a hierarchy gap condition, and uses sparse operators for stable large graph aggregation. Experiments on Cora, Citeseer, PubMed, and WikiCS show consistent gains over local baselines, with up to 2.0second best node classification accuracy on three of four datasets.**

***Keywords*—**Graph neural networks; adaptive context construction; directed graphs; hierarchy-aware learning; depth-limited BFS; link prediction

I. Introduction

Graph neural networks (GNNs) effectively convert graph interactions into representations that reflect both attributes and structure, yet standard local message passing is poorly matched to sparse directed graphs with hierarchical structure. A node may also play a different role depending on whether it is structurally upstream or downstream. Standard message passing is not tailored for this case. It is less reliable when the graph contains long range dependencies and role differences that are tied to direction.

Expanding the search space can recover distant context, but unconstrained expansion introduces noisy shortcuts, weakens hierarchy, and increases cost. On larger sparse graphs, the same process also increases the computational burden and makes the model harder to train. The key challenge is therefore to retain useful long range context without sacrificing structural fidelity or scalability.

This paper addresses that problem by adaptive context construction.The model keeps the directed graph for hierarchy estimation, restricts topology search through a constrained BFS, and only retains context that remains consistent with the inferred hierarchy. Sparse operators further keep the method practical on large graphs.

To summarize, the contributions of this paper are as follows:

1) We show that hierarchy estimation depends on preserving the directed graph.
2) We introduce a constrained adaptive context mechanism based on depth limited BFS.
3) We demonstrate that sparse operators keep AHGNN scalable to graphs with over 200K edges without sacrificing accuracy, validated across four benchmarks on node classification and link prediction.

II. Related work

Most GNNs, including GCN [1] and GAT [2], rely on local message passing. Methods such as APPNP [7], GPR-GNN [8], and FAGCN [9] enlarge or adapt propagation, but broader propagation is helpful only when irrelevant structure is filtered.

Hierarchy aware learning offers a stronger prior for directed graphs. SpringRank based AHGNN [10] uses hierarchy to guide context, but its unconstrained topology search remains noisy and costly. Pooling methods [11]–[13], graph transformers [14]–[18], scalable GNNs [19], and directed models such as MagNet [20] address complementary aspects of global structure, efficiency, or direction. Our method combines these motivations by preserving direction while constraining long range expansion with explicit depth and hierarchy gap rules.

III. Methodology

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote a graph with adjacency matrix A. Let $s = [s_1, \ldots, s_N]^T \in \mathbb{R}^N$ be the vector of SpringRank scores. The model estimates s by minimizing

$$H(s) = \sum_{i,j} A_{ij} \frac{1}{2}(s_i - s_j - 1)^2 + \frac{\alpha}{2} \sum_i s_i^2 \qquad (1)$$

α is a regularization parameter. Setting the derivative to zero yields

$$\left[D^{in} + D^{out} - (A + A^T) + \alpha I\right] s = (D^{out} - D^{in})\mathbf{1}, \quad (2)$$

where D^{in} and D^{out} are the diagonal in-degree and out-degree matrices and $\mathbf{1} \in \mathbb{R}^N$ is the all-ones vector. Although $A + A^T$ appears in the coefficient matrix, the graph is not symmetrized before hierarchy estimation; this term follows from the optimality condition of directed SpringRank, while asymmetry is preserved by $(D^{out} - D^{in})\mathbf{1}$ computed from the original A. Throughout the paper, s_i and s_j denote node scores, and edge wise discrepancy is defined separately. Symmetrizing the graph before ranking would flatten the hierarchy and weaken the context signal.

A. Hierarchical Estimation on Directed Graphs

The hierarchy stage keeps the original asymmetric adjacency matrix $A \neq A^T$. For each directed edge (i, j), define the residual

$$r_{ij} = s_i - s_j - 1 \qquad (3)$$

TABLE I
IMPACT OF BFS DEPTH (k) ON AHGNN PERFORMANCE

Task	Depth	Cora	PubMed	WikiCS
Node Class. (Acc %)	$k=1$	82.3	79.2	76.8
	$k=2$	**83.2**	**80.2**	**77.5**
	$k=3$	81.1	78.6	OOM
	$k=4$	78.5	76.4	OOM
	$k=5$	75.3	OOM	OOM
Link Pred. (AUC %)	$k=1$	94.1	97.0	95.5
	$k=2$	**95.6**	**98.5**	**97.2**
	$k=3$	93.5	96.2	OOM
	$k=4$	90.2	93.8	OOM
	$k=5$	88.4	OOM	OOM

which measures deviation from the ideal relation $s_i - s_j = 1$. Larger s_i indicates a more upstream node. We also define hierarchical depth as

$$d_i = -s_i, \tag{4}$$

so larger d_i indicates a more downstream node.

To measure node-level hierarchy fit, define

$$\psi_i^{out} = \frac{1}{2}\sum_j A_{ij} r_{ij}^2, \qquad \psi_i^{in} = \frac{1}{2}\sum_j A_{ji} r_{ji}^2, \tag{5}$$

$$\psi_i = \psi_i^{out} + \psi_i^{in} + \frac{\alpha}{2} s_i^2. \tag{6}$$

Here ψ_i^{out} and ψ_i^{in} measure mismatch from outgoing and incoming edges, while the regularization term stabilizes weakly connected nodes. Because A is asymmetric, the two terms are generally different.

High score nodes act as upstream authorities. The remaining issue is how to connect them to ordinary nodes without adding noisy links.

B. Depth Limited BFS

The key is to restrict the search space. Naive BFS grows rapidly, introduces noisy edges, and degrades performance when k becomes too large.

We therefore use a hop budget and a rank-gap condition. A shortcut is kept only if it is reached within depth k and satisfies a minimum hierarchy gap τ. As shown in Table I, $k = 2$ gives the best trade-off: $k = 1$ misses useful context, whereas $k = 3$ adds topological noise on Cora and PubMed and causes OOM on WikiCS.

Beyond depth control, the rank gap condition decides which long range edges remain consistent with the global hierarchy.

$$\mathcal{S}(v) = \{h \in \mathcal{H} \mid s_h - s_v \geq \tau\} \tag{7}$$

$\mathcal{H}$ denotes the top-p percentage of nodes ranked by SpringRank score; we fix $p = 10\%$ across datasets. τ is the hierarchy gap threshold that suppresses weak shortcuts.

For candidate authority h and target node v, define

$$\Delta(h, v) = s_h - s_v - \tau. \tag{8}$$

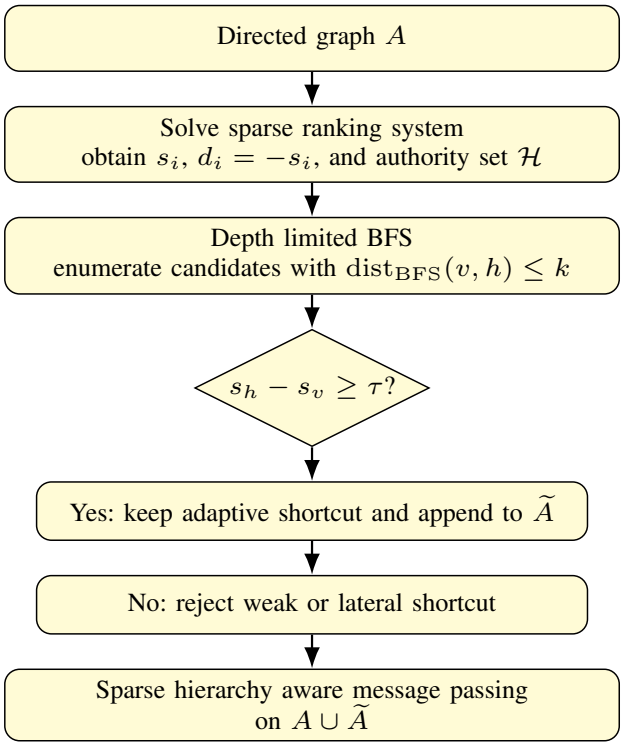


Fig. 1. Topology evolution flow for adaptive context construction.

A candidate edge is kept only when $\Delta(h, v) \geq 0$ and the BFS constraint also holds, yielding

$$\widetilde{A}_{vh} = \mathbf{1}\{\mathrm{dist}_{\mathrm{BFS}}(v, h) \leq k,\ h \in \mathcal{H},\ \Delta(h, v) \geq 0\}. \tag{9}$$

Bounding the search keeps candidate edges under control, especially on larger graphs. In practice, k sets the receptive field and τ filters lateral or weak shortcuts.

C. Hierarchy Aware Message Passing

Once the context graph is fixed, node representations are updated with hierarchy aware attention over the original neighborhood and retained context edges.

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \mathcal{S}(v)} \alpha_{vu} \mathbf{W} \mathbf{h}_u^{(l)}\right) \tag{10}$$

The attention coefficient jointly scores local and adaptive edges:

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_v \| \mathbf{W}\mathbf{h}_u] + \gamma \cdot \mathbb{1}[u \in \mathcal{S}(v)]))}{\sum_{w \in \mathcal{N}(v) \cup \mathcal{S}(v)} \exp(\dots)} \tag{11}$$

where $\mathbb{1}[\cdot]$ is the indicator function and γ is a learnable gate for adaptive context edges.

To remain practical on large sparse graphs, attention is evaluated only on active edges rather than on a dense $N \times N$ matrix, reducing computation from $\mathcal{O}(N^2)$ to $\mathcal{O}(|\mathcal{E}| + |\mathcal{S}|)$.

D. Time and Space Complexity

Let $N = |\mathcal{V}|$, $M = |\mathcal{E}|$, and $\widetilde{M} = |\widetilde{\mathcal{E}}|$. Solving Equation (2) requires

$$\mathcal{O}(T_{\mathrm{rank}}(M + N)) \tag{12}$$

TABLE II
SUMMARY OF DATASET STATISTICS

Dataset	Nodes	Edges	Features	Classes
Cora	2,708	5,429	1,433	7
Citeseer	3,327	4,732	3,703	6
PubMed	19,717	44,338	500	3
WikiCS	11,701	216,123	300	10

TABLE III
HYPERPARAMETER CONFIGURATION FOR AHGNN

Hyperparameter	Value
Hidden Dimension	8
Attention Heads	8
Top-p Ratio for $\mathcal{H}$	10%
Dropout Rate	0.6
Learning Rate	0.005
Weight Decay	5×10^{-4}
Training Epochs	1000
Early Stopping Patience	200
Optimizer	Adam

TABLE IV
NODE CLASSIFICATION ACCURACY (%) OVER 5 INDEPENDENT RUNS

Model	Cora	Citeseer	PubMed	WikiCS
MLP	55.1 ± 1.2	46.5 ± 1.5	71.4 ± 0.8	71.5 ± 0.9
GCN	81.5 ± 0.6	70.3 ± 0.8	79.0 ± 0.4	77.2 ± 0.5
GAT	81.8 ± 0.7	72.3 ± 0.7	78.5 ± 0.5	76.5 ± 0.6
APPNP	**83.3 ± 0.5**	71.8 ± 0.6	79.7 ± 0.4	77.0 ± 0.5
AHGNN	83.2 ± 0.4	72.3 ± 0.5	**80.2 ± 0.3**	**77.5 ± 0.4**

†On Citeseer, AHGNN and GAT achieve the same mean (72.3%).

time and $\mathcal{O}(M+N)$ space, where T_{rank} is the number of solver iterations. With average branching factor $\bar{b}$, depth limited BFS explores at most $\mathcal{O}\left(|\mathcal{H}| \sum_{\ell=1}^{k} \bar{b}^{\ell}\right)$ candidates, while the hierarchy gap filter retains only $\widetilde{M}$ edges in practice.

Each AHGNN layer then runs in $\mathcal{O}((M+\widetilde{M})d)$ time and $\mathcal{O}(M+\widetilde{M}+Nd)$ space, compared with $\mathcal{O}(N^2 d)$ time and $\mathcal{O}(N^2)$ memory for dense attention.

IV. RESULTS

We evaluate AHGNN on Cora, Citeseer, PubMed, and WikiCS (Table II). Node classification uses standard Planetoid splits for citation networks and official WikiCS masks. Link prediction uses a standard split with 10% test edges and 5% validation edges. We compare against MLP, GCN, GAT, and APPNP over 5 runs and report mean ± standard deviation; Table III lists the hyperparameters.

We evaluate node classification (Table IV, Figure 2) and link prediction (Table V, Figure 3).

For node classification, local aggregation is already strong on Cora and Citeseer, so gains are modest. AHGNN is most helpful on larger graphs, where unconstrained expansion would over smooth features: it reaches 80.2% on PubMed and 77.5% on WikiCS. On Cora, AHGNN (83.2%) is statistically comparable to APPNP (83.3%).

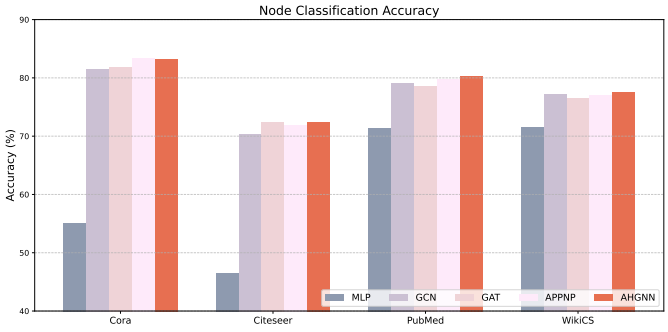


Fig. 2. Node classification accuracy comparison across Cora, Citeseer, PubMed, and WikiCS.

TABLE V
LINK PREDICTION AUC (%) OVER 5 INDEPENDENT RUNS

Model	Cora	Citeseer	PubMed	WikiCS
MLP	74.3 ± 1.5	70.5 ± 1.8	80.1 ± 1.2	75.0 ± 1.4
GCN	90.4 ± 0.8	89.4 ± 1.1	96.4 ± 0.5	90.2 ± 0.7
GAT	93.6 ± 0.6	94.0 ± 0.9	96.8 ± 0.4	94.6 ± 0.6
APPNP	92.1 ± 0.7	95.1 ± 0.8	97.4 ± 0.5	95.8 ± 0.5
AHGNN	**95.6 ± 0.4**	**96.8 ± 0.5**	**98.5 ± 0.2**	**97.2 ± 0.3**

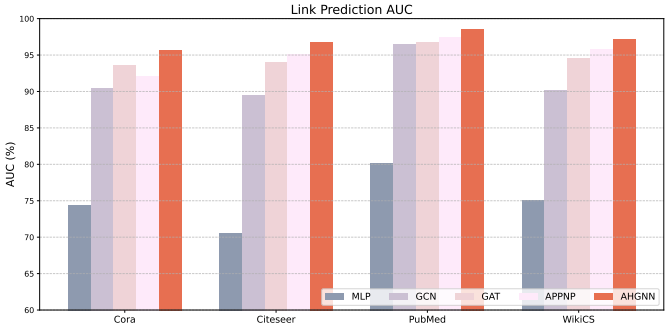


Fig. 3. Link prediction AUC comparison across Cora, Citeseer, PubMed, and WikiCS.

Link prediction benefits more because it depends on directional structure. By preserving direction in hierarchy estimation and context expansion, AHGNN outperforms all baselines, with AUC gains of 2.0%, 1.7%, 1.1%, and 1.4% over the strongest competing baseline on Cora, Citeseer, PubMed, and WikiCS.

A. Ablation on edge direction

We ablate edge direction by symmetrizing the graph ($A_{undir} = A + A^T$) before hierarchy estimation. Substituting $D^{in} = D^{out}$ into Equation (2) forces the right hand side to zero, flattening the hierarchy. Table VI shows that this undirected variant collapses to local baseline behavior, especially for link prediction.

TABLE VI
ABLATION: DIRECTED VS. UNDIRECTED HIERARCHY OVER 5 INDEPENDENT RUNS

Model Variant	Cora	Citeseer	PubMed	WikiCS
Node Classification (Accuracy %)				
Undirected ($A + A^T$)	81.8 ± 0.7	72.3 ± 0.7	78.5 ± 0.5	76.5 ± 0.6
Directed (AHGNN)	**83.2 ± 0.4**	72.3 ± 0.5	**80.2 ± 0.3**	**77.5 ± 0.4**
Link Prediction (AUC %)				
Undirected ($A + A^T$)	93.6 ± 0.6	94.0 ± 0.9	96.8 ± 0.4	94.6 ± 0.6
Directed (AHGNN)	**95.6 ± 0.4**	**96.8 ± 0.5**	**98.5 ± 0.2**	**97.2 ± 0.3**

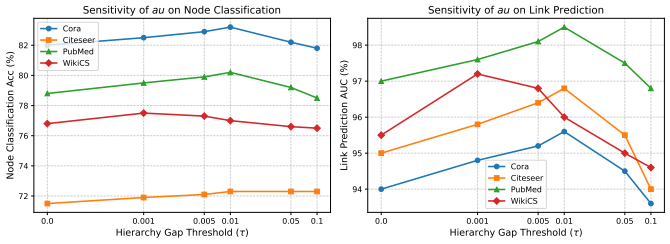


Fig. 4. Sensitivity analysis of the hierarchy gap threshold τ on Node Classification and Link Prediction tasks.

B. Parameter Sensitivity: Hierarchy Gap Threshold τ

We evaluate $\tau \in \{0.0, 0.001, 0.005, 0.01, 0.05, 0.1\}$. As shown in Figure 4, $\tau = 0.0$ keeps noisy lateral connections, while overly large thresholds reject valid context and regress toward local baselines. A small threshold works best, although its exact value depends on graph density.

Link prediction is more sensitive to τ because false positive shortcuts directly corrupt topology, whereas node classification remains relatively stable near the effective range.

V. CONCLUSION

AHGNN improves hierarchy aware graph learning by preserving direction, constraining context expansion with depth limited BFS and hierarchy gap filtering, and using sparse attention for scalable aggregation. Across four benchmarks, these choices reduce noisy shortcuts, improve link prediction most strongly, and remain effective on large sparse graphs. Future work can study adaptive depth selection and stronger sparse kernels.

REFERENCES

[1] T. N. Kipf and M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[3] M. Jin, H. Y. Koh, Q. Wen, D. Zambon, C. Alippi, G. I. Webb, et al. A Survey on Graph Neural Networks for Time Series: Forecasting, Classification, Imputation, and Anomaly Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10466–10485, 2024. doi: 10.1109/TPAMI.2024.3443141.

[4] C. Chen, Y. Wu, Q. Dai, H.-Y. Zhou, M. Xu, and S. Yang. A Survey on Graph Neural Networks and Graph Transformers in Computer Vision: A Task-Oriented Perspective. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10297–10318, Dec. 2024, doi: 10.1109/TPAMI.2024.3445463.

[5] Z. Zhang, P. Cui, and W. Zhu. Deep Learning on Graphs: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 249–270, 2022. doi: 10.1109/TKDE.2020.2981333.

[6] X. Wang, D. Bo, C. Shi, S. Fan, Y. Ye, and P. S. Yu. A Survey on Heterogeneous Graph Embedding: Methods, Techniques, Applications and Sources. *IEEE Transactions on Big Data*, vol. 9, no. 2, pp. 415–436, 2023. doi: 10.1109/TBDATA.2022.3177455.

[7] J. Gasteiger, A. Bojchevski, and S. Günnemann. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[8] E. Chien, J. Peng, P. Li, and O. Milenkovic. Adaptive Universal Generalized PageRank Graph Neural Network. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[9] D. Bo, X. Wang, C. Shi, and H. Shen. Beyond Low-Frequency Information in Graph Convolutional Networks. In *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 35, no. 5, pp. 3950–3957, 2021.

[10] D. Wu, H. Wu, and J. Li. Hierarchy-Aware Adaptive Graph Neural Network. *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 1, pp. 365–378, 2025.

[11] H. Gao and S. Ji. Graph U-Nets. In *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97, pp. 2083–2092, 2019.

[12] Z. Zhang, J. Bu, M. Ester, J. Zhang, C. Yao, Z. Yu, and C. Wang. Hierarchical Graph Pooling with Structure Learning. *arXiv preprint* arXiv:1911.05954, 2019.

[13] Z.-P. Li, S.-G. Wang, Q.-H. Zhang, Y.-J. Pan, N.-A. Xiao, J.-Y. Guo, et al. Graph Pooling for Graph-Level Representation Learning: A Survey. *Artificial Intelligence Review*, vol. 58, article 45, 2025, doi: 10.1007/s10462-024-10949-2.

[14] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu. Do Transformers Really Perform Badly for Graph Representation? In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 28877–28888, 2021.

[15] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini. Recipe for a General, Powerful, Scalable Graph Transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 14501–14515, 2022.

[16] Q. Wu, W. Zhao, C. Yang, H. Zhang, F. Nie, H. Jiang, Y. Bian, and J. Yan. SGFormer: Simplifying and Empowering Transformers for Large-Graph Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 64753–64773, 2023.

[17] C. Sun, C. Li, X. Lin, Z. Zheng, M. Meng, R. Gu, and Z. Wang. Attention-based graph neural networks: a survey. *Artificial Intelligence Review*, vol. 56, Suppl. 2, pp. 2263–2310, 2023, doi: 10.1007/s10462-023-10577-2.

[18] C. Yuan, K. Zhao, E. E. Kuruoglu, L. Wang, T. Xu, W. Huang, D. Zhao, et al. A Survey of Graph Transformers: Architectures, Theories and Applications. *arXiv preprint* arXiv:2502.16533, 2025.

[19] M. Ding, T. Rabbani, B. An, E. Wang, and F. Huang. Sketch-GNN: Scalable Graph Neural Networks with Sublinear Training Complexity. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 2930–2943, 2022.

[20] X. Zhang, Y. He, N. Brugnone, M. Perlmutter, and M. Hirn. MagNet: A Neural Network for Directed Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 27003–27015, 2021.